Implement image augmentation using Generative Adversarial Networks (GANs) on a small animal dataset

To implement a basic Generative Adversarial Network for generating additional animal images using a small animal dataset.

Algorithm

1. Import TensorFlow, NumPy and Matplotlib.
2. Load the CIFAR-10 image dataset.
3. Select animal categories such as birds, cats, deer, dogs, frogs and horses.
4. Combine the selected animal images into a training dataset.
5. Normalize image pixel values to the range -1 to 1.
6. Build a generator network that converts random noise into images.
7. Build a discriminator network that identifies real and generated images.
8. Define binary cross-entropy loss and Adam optimizers.

9. Train the discriminator using real and generated images.
10. Train the generator to produce images that fool the discriminator.
11. Repeat the training process for multiple epochs.
12. Generate new animal-like images and display them using Matplotlib.

Program

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
tf.keras.utils.set_random_seed(36)
# Load CIFAR-10 dataset
(x_train, y_train), _ = tf.keras.datasets.cifar10.load_data()
# Animal class IDs:
# Bird=2, Cat=3, Deer=4, Dog=5, Frog=6, Horse=7
animal_ids = [2, 3, 4, 5, 6, 7]
mask = np.isin(
y_train.flatten(),
animal_ids
)
images = x_train[mask][:3000].astype("float32")
# Normalize
images = (images - 127.5) / 127.5
noise_dim = 100
# Generator
generator = models.Sequential([
layers.Input(shape=(noise_dim,)),
layers.Dense(8 * 8 * 256, use_bias=False),
layers.BatchNormalization(),
layers.LeakyReLU(0.2),
layers.Reshape((8, 8, 256)),
layers.Conv2DTranspose(
128, 4,
strides=2,

padding='same',
use_bias=False
),
layers.BatchNormalization(),
layers.LeakyReLU(0.2),
layers.Conv2DTranspose(
3, 4,
strides=2,
padding='same',
activation='tanh'
)
])
# Discriminator
discriminator = models.Sequential([
layers.Input(shape=(32, 32, 3)),
layers.Conv2D(64, 4, strides=2, padding='same'),
layers.LeakyReLU(0.2),
layers.Dropout(0.3),
layers.Conv2D(128, 4, strides=2, padding='same'),
```

```
layers.LeakyReLU(0.2),
layers.Flatten(),
layers.Dense(1, activation='sigmoid')
])
# Loss and optimizers
bce = tf.keras.losses.BinaryCrossentropy()
g_opt = tf.keras.optimizers.Adam(1e-4)
d_opt = tf.keras.optimizers.Adam(1e-4)
# Training function
@tf.function
def train_step(real):
batch = tf.shape(real)[0]
noise = tf.random.normal([batch, noise_dim])
with tf.GradientTape() as d_tape, \
tf.GradientTape() as g_tape:
fake = generator(noise, training=True)

real_out = discriminator(real, training=True)
fake_out = discriminator(fake, training=True)
d_loss = (
bce(tf.ones_like(real_out), real_out) +
bce(tf.zeros_like(fake_out), fake_out)
)
g_loss = bce(
tf.ones_like(fake_out),
fake_out
)
d_grads = d_tape.gradient(
d_loss,
discriminator.trainable_variables
)
g_grads = g_tape.gradient(
g_loss,
generator.trainable_variables
)
d_opt.apply_gradients(
zip(d_grads, discriminator.trainable_variables)
)
g_opt.apply_gradients(
zip(g_grads, generator.trainable_variables)
)
return d_loss, g_loss
# Prepare dataset
dataset = (
tf.data.Dataset.from_tensor_slices(images)
.shuffle(3000)
.batch(64)
)
# Train GAN
for epoch in range(5):
d_values = []
g_values = []
for batch in dataset:
dl, gl = train_step(batch)

d_values.append(float(dl))
g_values.append(float(gl))
print(
"Epoch", epoch + 1,
"D Loss:", np.mean(d_values),
"G Loss:", np.mean(g_values)
)
# Generate images
noise = tf.random.normal([9, noise_dim])
generated = generator(
noise,
training=False
).numpy()
generated = (generated + 1) / 2
# Display generated images
fig, axes = plt.subplots(3, 3, figsize=(6, 6))
for ax, image in zip(axes.flat, generated):
ax.imshow(np.clip(image, 0, 1))
```

```
ax.axis('off')
plt.tight_layout()
plt.show()
```

Output of Program

```
Epoch 1 D Loss: ... G Loss: ...
Epoch 2 D Loss: ... G Loss: ...
Epoch 3 D Loss: ... G Loss: ...
Epoch 4 D Loss: ... G Loss: ...
Epoch 5 D Loss: ... G Loss: ...
```

Generated animal-like images are displayed in a 3 × 3 image grid.

Result

Thus, a basic Generative Adversarial Network was implemented using selected CIFAR-10 animal images. The generator learned to produce additional synthetic animal-like images for image augmentation.